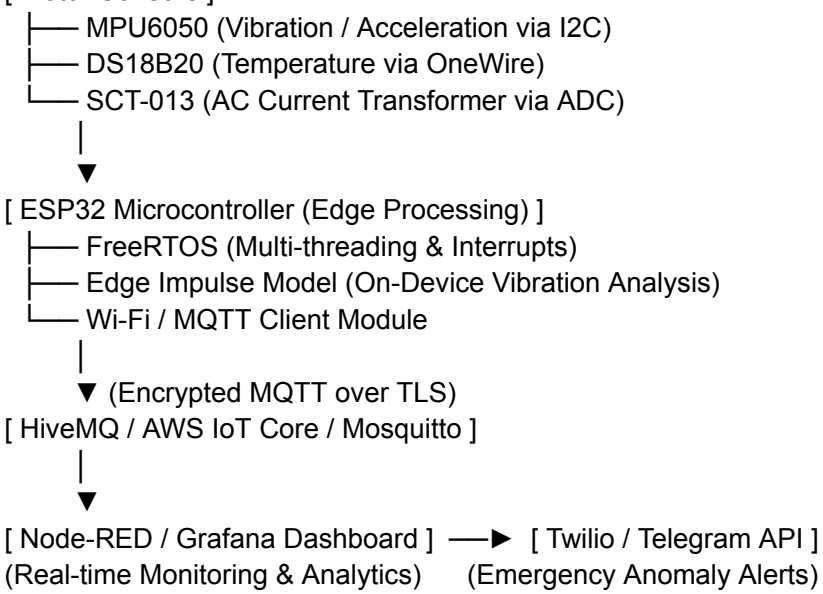
Here is a complete, production-ready final year project proposal tailored for an Embedded Systems & IoT degree. It balances core hardware concepts (firmware, interrupts, power optimization) with cloud infrastructure (MQTT, dashboards, data processing).

Final Year Project Proposal
Project Title
Edge-AI & IoT Predictive Maintenance System for Industrial Motors using ESP32

1. Problem Statement
In industrial environments, unexpected motor failures lead to costly downtime. Traditional maintenance relies on manual checks or fixed schedules. This project provides an end-to-end IoT solution that monitors motor health (vibration, temperature, and current draw) in real-time, predicts mechanical or electrical anomalies directly on the edge microcontroller using lightweight machine learning, and streams telemetry to a cloud dashboard.

2. System Architecture & Block Diagram


3. Key Hardware & Software Requirements
Hardware Components
 * Microcontroller: ESP32-WROOM-32 (Dual-core, Wi-Fi/BLE enabled).
 * Vibration/Motion Sensor: MPU6050 (3-axis Accelerometer + Gyroscope).
 * Temperature Sensor: DS18B20 (Waterproof digital probe).
 * Current Sensor: SCT-013 Non-invasive AC Current Transformer + ADS1115 (16-bit ADC for precision).
 * Actuators/Safety: Relay module (to auto-cutoff motor power during critical failure), status LEDs, and a 16x2 I2C LCD / OLED.
 * Target Load: Small 12V DC motor or 220V AC single-phase bench motor.
Software & Cloud Stack
 * Firmware Development: ESP-IDF or Arduino IDE with FreeRTOS.
 * Edge ML Engine: Edge Impulse (Train a TinyML model for anomaly detection using Fast Fourier Transform [FFT] data).

* Broker & Cloud: HiveMQ Cloud or AWS IoT Core (MQTT Protocol).
* Visualization/Control: Node-RED or Grafana dashboard.
* Notification System: Telegram Bot API or Twilio API (SMS / WhatsApp push alerts).

4. Step-by-Step Implementation Workflow

1. Hardware Interfacing & Data Collection

Sensors to ESP32

* Wire MPU6050 via I2C (SCL/SDA), DS18B20 via OneWire with a 4.7\text{ k}\Omega pull-up resistor, and the SCT-013 current sensor to an analog pin/ADS1115.
* Write bare-metal sensor drivers with sample rates set to 100\text{ Hz} for vibration data.

2. TinyML Edge Anomaly Training

Edge Impulse Integration

* Capture raw vibration data under three conditions: Normal Operation, Unbalanced Load (imbalance attached to motor shaft), and Bearing Wear (simulated friction).
* Export accelerometer time-series data to Edge Impulse.
* Train a Spectral Analysis (FFT) + K-Means/Neural Network model to classify anomalous vibration patterns locally without needing cloud connection.

3. Firmware Development & FreeRTOS

Dual-Core Execution

* Core 0: Handles high-speed sensor sampling and runs the Edge Impulse model inference every 2 seconds.
* Core 1: Manages Wi-Fi reconnect logic, processes incoming/outgoing MQTT JSON payloads, and handles LCD updates.

4. Cloud & Dashboard Setup

MQTT Data Pipeline

* Establish secure MQTT connection using SSL/TLS.
* Publish payloads under factory/motor1/telemetry with fields: {"temp": 42.5, "vibration_status": "NORMAL", "current_A": 1.2}.
* Build a Node-RED dashboard displaying live graphs for temperature, current, and health status logs.

5. Safety & Alerting Mechanisms

Fail-safe Controls

* Implement an over-current (>1.5\times rated) or high-temperature threshold (>70^\circ\text{C}).
* Trigger the relay module to safely isolate the motor on fault detection.
* Push instant warning messages to Telegram or WhatsApp.

5. Final Report & Viva Submission Highlights

To impress your project reviewers, make sure to document these technical details:

* Power & Bandwidth Efficiency: Explain how processing ML models on the edge (ESP32) saves cloud bandwidth compared to streaming continuous raw waveform data.
* Real-time Task Scheduling: Include a flowchart showing how FreeRTOS mutexes and queues prevent race conditions between sensor reading tasks and cloud publishing tasks.
* Accuracy Metrics: Include a confusion matrix showing the classification accuracy of your Edge ML model across different failure modes.